

FACULDADE DE CIÊNCIAS DA UNIVERSIDADE DO PORTO

Departamento de Ciência de Computadores

COMP — Compiladores

Trabalho Prático 2 — Geração de Código Intermédio e MIPS

Grupo: 6

Conteúdo

COMP — Compiladores	1
1. Introdução	1
2. Método	1
3. Tabela de Símbolos e Gestão de Memória.....	2
4. Representação Intermédia (IR)	2
5. Geração de Código MIPS.....	3
5.1 Gestão de Temporários (TempMap)	3
5.2 Tradução de Instruções.....	3
5.3 Estruturas de Controlo e I/O	3
6. Compilação e Execução	4
7. Conclusão	4

1. Introdução

Esta segunda etapa do projeto da unidade curricular de Compiladores teve como objetivo acrescentar ao analisador sintático, desenvolvido na primeira fase, as capacidades de análise semântica e geração de código. O produto final é um compilador capaz de processar um subconjunto da linguagem Ada, gerando código **Assembly MIPS** (32-bit) pronto a ser executado no simulador MARS, passando por uma representação intermédia linear.

2. Método

A solução segue uma arquitetura modular em pipeline, onde cada fase transforma a representação do programa:

1. Análise Sintática e Semântica:
 - O **Bison** constrói a **Árvore Sintática Abstrata** (AST) e invoca a gestão da Tabela de Símbolos para validar declarações.
2. Geração da Representação Intermédia (IR):
 - A estrutura hierárquica da AST é linearizada numa lista de instruções de "Código de Três Endereços" (TAC).
3. Geração de Código Máquina:
 - A lista de código intermédio é traduzida para instruções nativas MIPS, utilizando uma estratégia de gestão de memória baseada na pilha (*Stack-Based Allocation*).

3. Tabela de Símbolos e Gestão de Memória

A **Tabela de Símbolos**, implementada em **symtab.c**, é fundamental para a correta alocação de memória das variáveis do utilizador.

- **Estrutura de Dados:** Foi utilizada uma **lista simplesmente ligada** de estruturas **Symbol**. Esta é uma solução mais simples, mas perfeitamente eficiente para gerir as declarações.
- **Informação Armazenada:** Cada símbolo guarda o nome do identificador, o seu tipo (**SymbolType**, atualmente focado em **TYPE_INTEGER**) e, crucialmente, o seu **offset de memória**.
- **Cálculo de Offsets:** O compilador mantém um contador **current_stack_offset** (iniciado em 0). A cada nova declaração (**symbol_add**), o offset é decrementado em 4 bytes. Isto permite que o gerador de código saiba exatamente a posição relativa de cada variável face ao *Frame Pointer* (\$fp).

- **Validação:** São efetuadas verificações de declarações duplicadas na inserção e verificações de existência (`symbol_find`) sempre que uma variável é utilizada em expressões ou atribuições.

4. Representação Intermédia (IR)

O módulo `ir.c` implementa a abstração do código através de estruturas TAC (Three Address Code).

- **Estrutura Linear:** As instruções são armazenadas numa **lista simplesmente ligada**, onde cada nó contém a operação (TACOp), os operandos (`arg1`, `arg2`) e o destino (`result`).
- **Instruções Suportadas:** O conjunto de instruções cobre aritmética (TAC_ADD, TAC_SUB, etc.), lógica, atribuição (TAC_COPY), I/O (TAC_PRINT, TAC_READ) e controlo de fluxo (TAC_IFZ, TAC_GOTO, TAC_LABEL).
- **Temporários e Labels:** Para suportar a avaliação de expressões complexas e o controlo de fluxo, o sistema gera nomes únicos para variáveis temporárias (ex: `$t0`, `$t1...`) e rótulos de salto (ex: `L1`, `L2...`) através das funções auxiliares `make_temp` e `make_label`.

5. Geração de Código MIPS

- O gerador de código final, implementado em `mips_code.c`, traduz a lista de TAC para *Assembly*. A estratégia adotada privilegia a correção e a simplicidade, tratando a arquitetura MIPS como uma máquina de pilha.

5.1 Gestão de Temporários (TempMap)

Uma vez que o IR gera nomes de temporários como *strings* (ex: `"$t0"`), foi necessário criar uma estrutura auxiliar local no gerador, a **TempMap** (lista ligada). Esta estrutura associa cada temporário gerado pelo IR a um endereço físico na stack, situado abaixo das variáveis locais do utilizador. Isto garante que nunca esgotamos os registos físicos da CPU, permitindo compilar expressões de qualquer complexidade.

5.2 Tradução de Instruções

A tradução segue o padrão *Load-Operate-Store*:

1. **Carregamento:** Os operandos são carregados da memória (usando os *offsets* calculados) para registos temporários da CPU (`$t0`, `$t1`) via

instrução lw.

2. **Execução:** A operação MIPS correspondente é executada (ex: add, mul, div).
3. **Armazenamento:** O resultado é imediatamente guardado na pilha (sw) no *offset* correspondente à variável ou temporário de destino.

5.3 Estruturas de Controlo e I/O

- **Comparadores:** Operações relacionais (ex: $\geq$) são sintetizadas utilizando instruções slt (*Set Less Than*) combinadas com xori para inversão lógica quando necessário.
- **Fluxo:** O comando TAC_IFZ (*If Zero*) é traduzido diretamente para beqz, permitindo a implementação correta de ciclos WHILE e condicionais IF.
- **System Calls:** As operações de entrada e saída utilizam as *syscalls* do MIPS (código 1 para imprimir inteiros, 4 para strings e 5 para ler inteiros).

6. Compilação e Execução

O projeto é compilado através de um Makefile que automatiza a invocação do Flex, Bison e GCC. Para testar o código gerado:

1. Compilar o projeto: make
2. Gerar o assembly: ./compiler input.adb > output.asm
3. Executar: Abrir o ficheiro output.asm no simulador MARS e executar (com suporte a *input* na consola do simulador).

7. Conclusão

O compilador encontra-se completo e totalmente funcional, gerando código Assembly MIPS conforme os requisitos e validado no simulador MARS. A solução apresenta uma implementação robusta das fases de *front-end* e *back-end*, suportada por uma Representação Intermédia (TAC) que facilita a tradução de estruturas de controlo e operadores lógicos de curto-circuito.

Destaca-se, em particular, a estratégia de gestão de memória baseada na pilha (*stack-based*). Embora privilegie a simplicidade, esta abordagem

revelou-se eficaz para garantir a estabilidade do sistema, permitindo processar expressões complexas sem limitações de registos físicos. Em suma, o projeto cumpre todos os objetivos propostos, entregando uma ferramenta capaz de compilar corretamente o subconjunto da linguagem Ada.